Auditing Generated KPI Documentation Against Source Code: A Study of 28 KPI Pages

Meshaal Obaid Mouawad[a,*], Bryan Jones[a]

[a]Department of Electrical and Computer Engineering, Mississippi State University, Mississippi State, 39762, MS, USA

ARTICLE INFO

Keywords:
software documentation
program comprehension
provenance
traceability
key performance indicators
explanation fidelity
knowledge engineering

ABSTRACT

Software calculates key performance indicators (KPIs), but the documentation explaining those calculations may not accurately describe what the code does. Even when documentation links to the correct source code, its explanations may contain errors or omit conditions that affect the calculated result. We evaluated this problem using LEAP (Literate Programming for Automated KPI Extraction), a research software tool that extracts KPI information from source code and publishes source-linked documentation in an Interactive Bluebook.

We audited 28 generated KPI documentation pages using deterministic checking rules and assessments from three independent reviewers. The evaluation tested four candidate dimensions of documentation accuracy. The automated source-role checker detected only 2 of 16 errors identified by reviewer majority (12.5% recall). For condition disclosure, checker precision differed between two assessment criteria: 40% when source visibility counted as disclosure and 100% under the reader-facing criterion used by the checker. The latter result reflected agreement with that criterion, not independent proof of perfect detection. The study demonstrated that linking documentation to source code does not guarantee an accurate explanation. It also identified specific documentation errors and limitations of the automated rules used to detect them.

1. Introduction

Key performance indicators (KPIs) are quantifiable measures used to evaluate progress toward strategic and operational objectives. Their meaning connects a business definition of what should be measured with the calculation that produces the reported value. In software-based reporting, this connection includes the inputs, formula, and conditions implemented in source code. Understanding a KPI therefore requires access to both its business interpretation and the computational decisions that determine its value (Maté et al., 2017; Roldán-García et al., 2021). We investigated this connection by auditing 28 generated KPI documentation pages against their source code and comparing automated decisions with judgments from three independent reviewers. The study identified inaccurate explanations, traced recurring errors to the code that generated them, and measured which reviewer findings the automated checks detected.

In the manual workflow motivating this study, the business definition, software, and documentation are maintained separately. A business analyst specifies the indicator, a developer implements its calculation, and a separate reference document explains it to business readers. When a reported value is questioned or the code changes, the team must compare that document with the implementation to check which inputs, formula, and exceptional cases apply. The code produces the number; the document is meant to explain its meaning. Research on documentation maintenance and code–comment consistency shows why that connection must be maintained as software evolves (Aghajani et al., 2019; Wen et al., 2019; Xu et al., 2024).

LEAP (Literate Programming for Automated KPI Extraction) is a research software tool that scans source files for potential KPI calculations and generates documentation pages linked to the code. Its collection of pages is called the Interactive Bluebook; each page is called a dossier. In the evaluated 28-page set, each dossier BAJ: can show, or does it always show? Preferred: A dossier shows... presents a reported formula, source-code location, descriptions of code, safeguards, and review status. While the links help people find the source, they cannot guarantee that the generated descriptions are correct.

For a page combining a business formula, source excerpts, safeguards, and review information, the question pursued here is which discrepancies remain in the published explanations and how reliably the proposed checks identify them. We compared each page with its source, traced recurring discrepancies to generation logic, and used independently recorded judgments to assess both the proposed criteria and the checker. The worked example below makes this question concrete.

For a concrete example, consider Ethylene Production Yield. The ordinary formula divides the tons of ethylene produced by the tons of feedstock supplied, then multiplies by 100. However, the archived source returns zero whenever the feedstock input is zero or negative. LEAP publishes the ordinary ratio and retains a link to the code, but its Logic Safeguards section, the part intended to explain exceptional-input handling, does not tell the reader about that branch. Someone reading only the documented ratio cannot see what the software will report for those inputs. Figure 1 follows this example through LEAP, the audit, and the independent

*Corresponding author: Meshaal Obaid Mouawad
Email addresses: mm4922@msstate.edu (M.O. Mouawad); bjones@ece.msstate.edu (B. Jones)
ORCID(s): 0000-0002-1152-8324 (M.O. Mouawad); 0000-0002-5807-8797 (B. Jones)

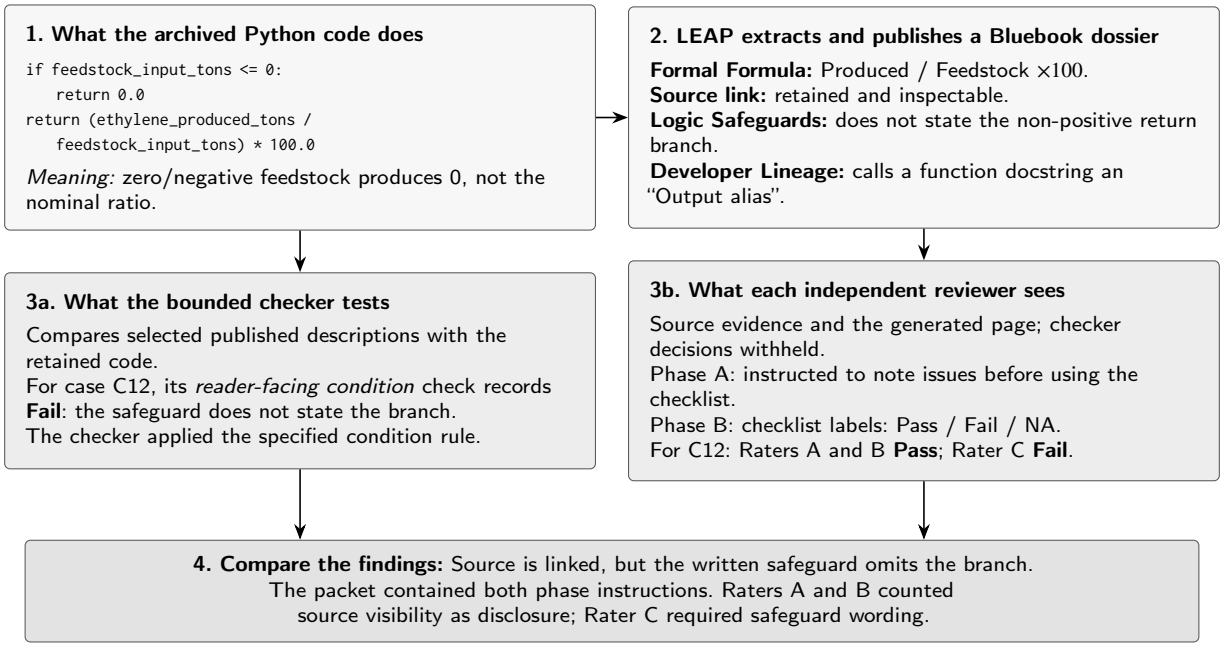


Figure 1: Schematic of the archived Ethylene Production Yield source, generated page, and recorded audit decisions. The checker assessed the condition using its reader-facing rule; reviewers assessed the supplied source and page independently of the checker outputs. Their condition labels differed over what counted as disclosure.

Table 1
Illustrative outputs of the archived Yield function for positive, zero, and negative feedstock inputs.

Ethylene produced (tons)	Feedstock input (tons)	Returned yield (%)
20	100	20
20	0	0
20	−10	0

reviewers. Table 1 gives illustrative outputs for the same function.

The source link makes the calculation available for inspection; the omitted branch shows why the explanation itself must also be checked. We call this correspondence between the published description and its linked source *explanation fidelity*. The audit considered source roles, behavior-changing conditions, preservation of marked conflicts, and wording about review status. Section 2 explains how prior work motivates these four questions.

Reviewer judgments supported source-role accuracy and condition disclosure as the two retained criteria. The source-role rule detected two of 16 reviewer-majority failures (12.5% recall). Condition precision was 40% or 100% depending on which of the two disclosure criteria identified in reviewer judgments were employed. We narrowed disagreement preservation to a LEAP-specific routing check and rejected the review-authority criterion. These findings distinguish agreement on what should be assessed from the coverage of its automated implementation.

These findings yielded three contributions:

1. an audit that compares generated KPI descriptions with their source code and traces discrepancies to the code that generated the documentation;
2. an independent assessment of four proposed criteria on 28 pages, retaining source-role accuracy and condition disclosure, narrowing disagreement to explicit-conflict routing, and rejecting review-authority consistency; and
3. reproducible comparisons showing which discrepancies remained after source links were verified, which reviewer findings the checker missed, and how its measured precision changed with the definition of disclosure.

2. Related Work and Basis of the Assessment Criteria

The literature leads to the present question through three related capabilities: connecting documentation to its source, checking what that documentation says, and recording evidence or unresolved disagreement. KPI research adds a domain-specific requirement: the implemented calculation should remain connected to the business meaning of the indicator. The following synthesis explains what these contributions establish and which evaluation question we carry into the LEAP case study.

2.1. Literate Programming and Documentation Generators

Connecting explanation to implementation has an established basis in literate programming and source-linked documentation. Knuth's WEB approach interleaves procedural code and narrative so that readers can follow a program's

rationale and structure alongside its executable components (Knuth, 1984). Documentation generators provide further mechanisms for publishing source-connected explanations. Doxygen incorporates structured documentation comments, Javadoc associates documentation with Java declarations, and rustdoc compiles and runs suitable code examples as documentation tests (Doxygen, n.d.; Oracle, n.d.; The Rust Project Developers, n.d.). BAJ: This work builds on the strengths of literate programming while also providing the ability to check the meaning of the established connections. The present study builds on source-connected documentation by evaluating whether generated explanations accurately describe the implementations to which they link.

Checking meaning requires a further comparison. Wen et al. (2019) documented several types of code–comment inconsistency and identified the need for detectors covering more types. Zhou et al. (2017) checked particular behavioral statements about API parameters and exceptions. Eghbali et al. (2026) went beyond consistency to examine whether documentation contains enough information to reconstruct equivalent function behavior. These contributions motivate testing both the accuracy of a generated explanation and the coverage of the procedure used to assess it. LEAP brings the business formula, source code, explanations, safeguards, and review status together on one KPI page. We examined whether the descriptions on that page accurately communicated the linked implementation.

2.2. From Source Connection to Accurate Behavioral Explanations

Keeping documentation available is the starting point. Aghajani et al. (2019) examined documentation-related reports and discussions and developed a taxonomy of problems affecting its quality. Looking specifically at source comments, Wen et al. (2019) studied code–comment changes and classified inconsistencies repaired by developers. They observed that available detectors covered particular error types and explicitly identified broader detection coverage as a research need. The resulting question is concrete: which kinds of inaccurate explanation can a checking method recognize, and which does it miss?

Two detection approaches illustrate different parts of that question. Zhou et al. (2017) combined analysis of API code and documentation to identify defects in parameter constraints and exception directives. Their comparisons show how a documented behavioral requirement can be checked against implementation evidence. Xu et al. (2024) addressed another source of difficulty: noisy labels in data used to learn code–comment inconsistency. Their confidence-learning method treats the quality of the comparison labels as part of the detection problem. Together, these studies motivate examining both the decision rule and the evidence used to judge its output.

Eghbali et al. (2026) investigated a stronger property than the absence of contradictory statements: whether documentation describes enough behavior to reconstruct an equivalent function. Their Documentary approach iteratively generated code from documentation and refined the explanation through equivalence checks. The authors explicitly identify test coverage and the focus on Python functions and docstrings as boundaries of their evaluation. For a generated KPI page, the related question concerns several published fields together: whether the source-role description, displayed formula, and safeguard explain the same retained calculation. These studies identify errors in explanations of code and its behavior. LEAP extends prior work by placing the source beside generated formulas, explanations, and safeguards; we compared those fields with the retained source. This comparison applied the two requirements motivated by the cited work: describe source elements correctly and state the conditions that qualify their behavior.

We take behavioral consistency and the need to assess detection coverage from these studies. With source evidence retained, we ask whether the published explanations correctly describe source elements and the conditions that affect their behavior. BAJ: This approach lets us ask a separate question: did the published explanation correctly describe the role of the source element and the conditions affecting its behavior?

2.3. From Traceable Evidence to Reviewable Disagreement

When two recorded formulas disagree, retaining access to both expressions allows the disagreement to be examined. Nuseibeh et al. (2001) developed an approach to managing known inconsistencies rather than requiring their immediate elimination. This motivates a separate question from formula correctness: does publication preserve the disagreement for review? Evidence models then help distinguish what a claim says from the information supporting it. SEPIO (Scientific Evidence and Provenance Information Ontology), an evidence model introduced by Brush et al., links claims to supporting or refuting evidence (Brush et al., 2016). For the related task of classifying evidence, Giglio et al. (2019) described ECO (Evidence and Conclusion Ontology) for recording evidence types and how a claim was made. These complementary contributions motivate examining both visible conflicts and the status communicated by the page. Motivated by these works, LEAP retains the source representations, conflict markers, and review status on the same page. It checks whether marked conflicts remained routed for review and whether certification wording agreed with that status. It also assesses reviewer agreement on both proposed criteria separately from the behavior of the corresponding automated rules.

Maintaining the connection to evidence is itself a research problem. Mäder and Gotel (2012) studied how traceability links can be maintained as software models change. Missier et al. (2013) described PROV, the World Wide Web Consortium's family of provenance specifications, for recording data, the activities that produced it, and the responsible people or systems; the associated constraints specify valid provenance records (World Wide Web Consortium, 2013). For database execution, Psallidas et al. (2023) reconstructed query dependencies from event logs in OneProvenance.

Sadiq et al. (2026) combined provenance, validation, publication, and access controls in ESDPKI (Enterprise Spatial Data Provenance Knowledge Infrastructure). These works establish ways to maintain, recover, and manage evidence connections. Our LEAP contribution distinguishes that capability from assessing the explanation written alongside an already available connection: a source link identifies the implementation behind a KPI, while the review status states how the page is classified. We examined the descriptions accompanying those records. This separates the existence of source and status records from the accuracy of what the page tells the reader.

2.4. From Generated Knowledge to Independent Assessment

Recent proposals make the quality of source-derived knowledge explicit. In a preprint, Gao (2026) proposed attaching executable behavior, evidence, and validation information to selected code units in ECK (Executable Code Knowledge). This makes the relationship between a generated representation and its supporting code part of the representation itself. In a complementary preprint, de Macedo and da Costa (2026) presented Reversa, which derives descriptions of legacy-software behavior to guide coding agents and retains confidence labels and gaps requiring review.

The Reversa authors explicitly report that their confidence index came from their own pipeline and that independent factual-accuracy evaluation remained future work. That stated boundary separates recording confidence from assessing correctness independently. It motivates the type of comparison used here: generated claims are checked against retained source evidence and separately recorded reviewer judgments. The present evaluation concerns LEAP pages and their audit rules.

Together, these studies motivate checking both the connection to source code and the accuracy of its explanation. Therefore, LEAP provides the page on which code, formula, safeguards, and review information can be inspected together. We evaluated the four proposed assessment criteria using reviewer agreement and measured how the implemented checker compared with the recorded judgments.

2.5. From KPI Definitions to the Present Audit Question

The domain-specific foundation concerns how a business definition reaches its implementation. Maté et al. (2017) linked strategic objectives and KPI definitions expressed in business terminology to the data used to evaluate them. Roldán-García et al. (2021) addressed KPI modelling, selection, and reasoning through an ontology-based approach. These contributions make indicator meaning and relationships explicit. Diamantini et al. (2025) surveyed this wider family of semantic indicator models, identifying shared meaning and alignment between definition and implementation as central concerns.

Once an indicator changes, consistency must also be maintained across its definitions and dependencies. Domínguez et al. (2025) addressed this problem in AFFECTK, the KPI-evolution framework introduced by Dominguez et al., organizing KPI evolution around traceability, consistency, and propagation of changes and evaluating a reasoning-based prototype. This contributes a way to manage changes in the indicator model. We examine the complementary publication step: when documentation is produced from an implementation, does the resulting explanation retain the relevant source role, condition, disagreement, and review information? Therefore, the LEAP framework provides a completed audit of generated KPI explanations: we located discrepancies against linked code, traced their causes, and compared automated checks with independent reviewer judgments. This distinguishes the usefulness of an assessment criterion from the performance of the rule used to implement it.

Taken together, the reviewed work supplies methods for connecting and managing evidence, checking particular behavioral claims, and representing KPI meaning. The contribution introduced by the LEAP framework is how to assess the explanations on a source-linked KPI page and establish the coverage of that assessment. We operationalized four candidate criteria, audited the 28-page output, traced recurring errors to the generator, and compared the rules with three raters. Table 2 summarizes the literature-to-check relationship. The following research questions separate finding and explaining the discrepancies from evaluating agreement and detection performance.

3. Research Questions and Method

The completed study addressed the following research questions:

RQ1 Which discrepancies between source evidence and published KPI explanations are exposed in the full 28-dossier LEAP output, and which generator mechanisms explain the repeated manifestations?

RQ2 Which discrepancies in published KPI knowledge persist or vary across an earlier project state and alternative input configurations?

RQ3 Which candidate explanation-fidelity dimensions achieve reproducible independent reviewer agreement?

RQ4 What checker failure modes and criterion sensitivities appear when the released rules are compared with independently produced reviewer labels?

We addressed RQ1 by comparing published KPI explanations with their linked code and tracing recurring errors to generator logic. For RQ2, we inspected an earlier output and outputs from alternative inputs. Three independent reviewers then assessed the four proposed criteria for RQ3; RQ4 compared their recorded judgments with the automated rules. The following sections specify the artifacts, decisions, and comparison procedures used at each stage.

Table 2
Literature basis and implemented checks for the four proposed assessment criteria.

Assessment criterion	Established basis	Implemented LEAP check
Source-role accuracy	Code–text consistency and documentation equivalence (Wen et al., 2019; Xu et al., 2024; Eghbali et al., 2026)	Flag a published explanation that assigns a source element the wrong syntactic/semantic role.
Condition disclosure	Completeness of behavioral/API directives (Eghbali et al., 2026; Zhou et al., 2017)	For explicit zero/non-positive denominator guards, preserve the guarded variable and condition semantics.
Representation disagreement	Inconsistency management and conflicting evidence (Nuseibeh et al., 2001; Brush et al., 2016)	For source evidence carrying an explicit conflict marker, preserve the conflict in review routing.
Review-authority consistency	Evidence/assertion roles, provenance validity, and validation-governance separation (Giglio et al., 2019; World Wide Web Consortium, 2013; Sadiq et al., 2026)	On unresolved dossiers, flag bounded certification-oriented wording.

Table 3
Major claims, supporting evidence, and falsification conditions.

Claim	Supporting evidence	Falsification condition
Within the LEAP case study, source linkage can coexist with publication discrepancies.	Source-linked Yield and Gross Margin dossiers retain inspectable source evidence while direct comparison identifies role and condition discrepancies.	The claim fails if the retained evidence link is absent or direct source comparison removes the reported discrepancy.
Three parts of the LEAP generator produced the repeated documentation errors.	Source inspection traced source-role descriptions, safeguards, and certification wording to three shared parts of the generator.	The claim fails if rerunning the released audit changes the manifestations or if repeated manifestations map to different generator mechanisms.
Independent review differentiates the candidate dimensions.	Three reviewers assessed 112 page–criterion combinations; Fleiss' κ ranges from 0.553 to -0.285, and restricting the disagreement analysis to cases judged applicable by all three reviewers changed the result.	The claim fails if the preserved reviewer forms and deterministic aggregation reproduce uniform agreement across the four candidates.
The released source-role rule fails coverage, while condition-rule performance is criterion-sensitive.	Source-role recall is 12.5% against reviewer majority. On the 10 checker-eligible condition cases, precision is 40% under the source-visibility criterion and 100% under the reader-facing criterion; recall is 100% under both.	The claim fails if deterministic recomputation removes the source-role confusion matrix or the dual-criterion condition results.

3.1. Claims and falsification conditions

Table 3 links each main claim to its evidence and states what observation would overturn it.

3.2. Case-study system and publication pipeline

To make comparisons reproducible, we evaluated one fixed version of LEAP, its input files, and the KPI pages produced from them. We call this a *frozen evaluation snapshot*: a copy held unchanged so that the same inputs and outputs can be inspected again. It does not mean that LEAP normally stops changing or generating new documentation.

There were four distinct roles in the evaluation. LEAP generated the source-linked KPI pages. The checker applied predetermined rules to the source and published fields. The reviewers inspected the cases and recorded judgments. We compared those judgments with checker outputs and inspected the generator code to trace repeated discrepancies.

Figure 2 separates page generation from the checker, reviewer, and comparison steps. LEAP generates documentation from KPI source evidence. In this evaluated version, its implementation scans source files for KPI candidates, prepares formula and business-context fields, attaches source links and review information, and generates an HTML page for each selected KPI. The page contains source context, a displayed formula, descriptions of source elements, safeguards, ownership information, conflict markers, and a review status. Its Developer Lineage field describes what each linked source excerpt represents, such as a comment or a named output column. We compared each published explanation with its linked source, then examined the generator code responsible for the incorrect descriptions.

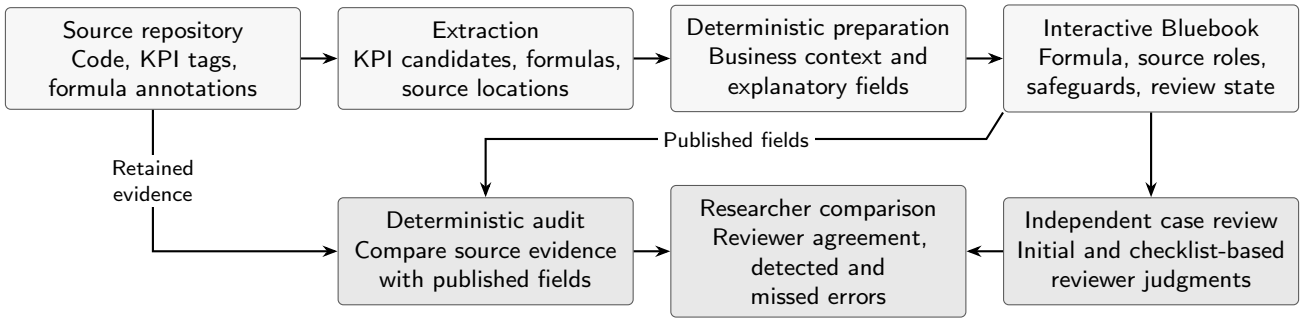


Figure 2: The evaluated LEAP workflow (upper row) and the audit and reviewer comparison (lower row). LEAP generated the pages; the checker and reviewers assessed them separately; the researchers compared their decisions. The archived generation log records deterministic detail preparation for all 28 pages.

3.3. Case-study artifacts and within-system design

We examined the 28 pages produced by the fixed evaluation version and used older and alternative-input outputs to see whether the same errors recurred. The main pages were held unchanged so every checker and reviewer evaluated the same material. The main dataset consists of 28 KPI dossier pages saved as HTML files (webpage documents) by the fixed LEAP evaluation version. The number 28 counts generated pages, not screenshots or separate experiments. A public artifact copy at commit `5e697b5b` (full identity in the supplement) matches all 28 submitted pages byte-for-byte, as verified using SHA-256 file hashes (content-based file identifiers). The build contains 9 dossiers labeled *Validated* and 19 labeled *Needs Review*. These are statuses recorded by LEAP, distinct from the reviewers' later assessments. These recorded states support the concrete LEAP certification-string analysis and also provide evidence for evaluating the broader review-authority candidate independently.

We compared the earlier archived output of 18 pages, including 6 *Validated* and 12 *Needs Review* pages, to determine whether the same errors recurred. Two further input configurations used the same generator: 38 scenario-fixture files produced 29 pages, and 8 challenge files produced 8. These comparisons tested whether the findings changed with the input configuration. We treated each generated page as a case and assessed its source descriptions, calculation conditions, conflict markers, and review-status wording.

External adapter check. We tested whether the source-role and condition checks could read another documentation format using four official FastAPI tutorial sources and their generated OpenAPI 3.1 descriptions at fixed upstream identities (FastAPI Project, 2026; OpenAPI Initiative, 2026). FastAPI is a Python framework for building application programming interfaces (APIs); OpenAPI is a machine-readable description of the API's inputs and constraints. Here, an *adapter* maps the other system's source and published fields into the comparisons used by the checker. These executions demonstrated adapter operation on another format; the independent reviewers assessed the LEAP pages.

3.4. Automated Checking Rules

A generated KPI page contains explanations of code, conditions that change a calculation, and information about unresolved differences and review status. We designed four questions to check these fields for particular mistakes. A *checking rule* is an instruction that compares specified information and returns a decision. The rules are *automated* because a program performs these comparisons using saved code and documentation fields, rather than asking a person to make each decision. We also asked human reviewers to inspect the pages, allowing us to measure which problems the program missed.

The checker applied four bounded candidate checks, each comparing specified source evidence with a published description. They targeted particular error patterns rather than every possible documentation failure. A *guard* here means an explicit source-code condition that changes how an exceptional input is handled. The reviewer comparison measured which discrepancies these specified checks detected or missed. The rules implemented specific checks motivated by the literature in Table 2. Prepared controls tested whether each rule returned its specified decision. Comparison with independent reviewers tested how many of their findings it detected. For source roles, it detected only 2 of 16 reviewer-majority failures.

1. **Source-role accuracy.** The *role* of a source element is what it does: a comment explains code, a data-type conversion changes representation, and an output alias names a result. A generated explanation is inaccurate when it assigns the wrong role to an element. Figure 3 gives a concrete example. The checker flags a failure when a comment or docstring (a function's embedded description) is described as executable code, or when a SQL `CAST` conversion type is labeled as an output alias (the name assigned to a result) instead of the actual alias. The rule reads pairs consisting of a source excerpt and its generated Developer Lineage description.

2. **Executable-guard disclosure.** A *guard* is a condition in source code that changes how the program handles an exceptional input. In the Yield example, the function returns zero if the input is zero or negative. This check asks

whether the published explanation tells the reader about that behavior. It covers explicit zero or non-positive denominator guards expressed as `NULLIF(variable,0)` (which returns a null value when the denominator variable equals zero) or an `if`-family comparison to zero. Preservation requires reader-facing safeguard text to communicate zero/non-positive handling and match the guarded variable to the denominator of the displayed formal formula (the equation printed in the dossier). A `<= 0` guard requires non-positive semantics; zero-only wording fails the rule.

3. **Representation-disagreement candidate.** A *representation disagreement* exists when two recorded descriptions of the same calculation conflict, for example the stated business formula and the implemented formula. The rule checks whether a known disagreement remains visible for review. The released rule covers pages with an explicit source conflict, mismatch, or inversion marker. It verifies that the disagreement remains visible through *Needs Review* status and an entry in the Review Queue (the list of items requiring a decision). We separately measured reviewer agreement on the broader disagreement criterion.

4. **Review-authority candidate.** *Review authority* concerns the approval status readers understand from a page's wording. On a page marked *Needs Review*, certification wording could imply that an unresolved calculation has already been approved. The released rule checks such pages for bounded certification-oriented wording such as "certified metric dossier," "certified calculation expression," or "print certified dossier." Reviewer agreement determined whether this criterion was retained.

The condition rule covered the specified zero and non-positive branches. It did not check near-zero numerical tolerances or the input domains of square-root, logarithmic, or inverse-trigonometric functions; those require separate tests (Section 6.5). The scripts produced the full guard inventory and line-level source-role findings. Table 2 records the literature basis of the four criteria.

3.5. Evaluated checker and conformance tests

First, we tested whether the automated checker followed its own rules on examples with known expected decisions. Such a *conformance test* asks whether a deliberately incorrect example is flagged and a correct one is accepted. Passing these tests demonstrates the specified behavior on those examples; it does not establish that the checker detects every mistake on the generated KPI pages.

The checker compares published roles and names with their source counterparts, requires a nonempty displayed denominator, and rejects missing, heading-only, or contradictory safeguard descriptions. The conformance tests below exercise these requirements through prespecified control outcomes.

We tested the checker against two sets of author-specified controls with expected labels stored separately from its output. Eighteen rule fixtures covered correct and incorrect source roles, missing or mismatched denominator information,

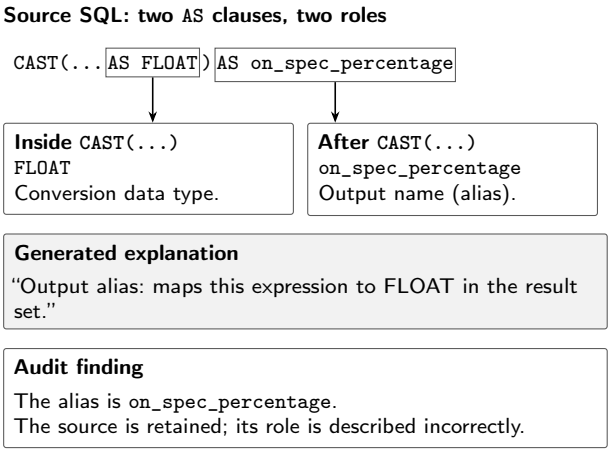


Figure 3: On-Spec Rate source-role error. The ellipsis omits arithmetic from the archived SQL statement. The two boxed `AS` clauses identify the conversion type and the output name; the generated Developer Lineage text confuses them.

omitted or contradictory safeguard text, conflict routing, certification wording, and applicability. Fourteen controls derived from the archived artifacts included five unchanged baselines, eight substitutions of extracted inputs, and one HTML mutation read again through the same page adapter. We compared each returned decision with its expected label. Matching the predefined expected labels (the control oracle) includes returning Fail when a control is deliberately defective. Across the 32 controls, the expected outcomes comprise 19 Fail, 11 Pass, and two N/A (not applicable) labels. These results establish conformance to the specified control cases. Independent comparison on the original dossiers reveals the coverage limitation: the source-role checker misses 14 of the 16 reviewer-majority failures.

3.6. Independent Review: Initial and Checklist-Based Phases

We separately asked three independent raters BAJ: better: experts? Business analysts? to evaluate the same KPI pages because automated rules can miss errors outside their predefined patterns. In Phase A, they recorded issues they noticed; in Phase B, they applied the four proposed assessment criteria. Their judgments provided a comparison for the checker's recorded decisions. The two phases were conducted in that order.

Three independent reviewers, designated *Rater A*, *Rater B*, and *Rater C* throughout this paper, evaluated the same 28 pages (C01–C28). The packets excluded checker outputs, manuscript result tables, the organizer mapping key, and other reviewers' responses. In Phase A, reviewers were instructed to record issues using their own judgment before applying the four-criterion checklist in Phase B. The packet contained the instructions for both phases. Phase B recorded Pass, Fail, or N/A with supporting evidence; N/A meant that the criterion was judged inapplicable.

Each of the 28 pages was assessed against four criteria, producing 112 page–criterion combinations. Three ratings

per combination produced 336 individual ratings. All 28 Phase-A case rows were returned by each reviewer. The original Phase-B responses contained 242 Pass/Fail ratings and 94 explicit N/A ratings (28.0%). The 94 N/A entries appeared as blanks in an intermediate analysis export: 40 of 84 condition ratings and 54 of 84 disagreement ratings. Restoring those entries to N/A reproduced all 336 original labels. We also calculated agreement on the cases for which all three reviewers supplied Pass or Fail, indicating common applicability.

A majority decision required at least two matching ratings for a page–criterion combination. One condition case had a three-way split, leaving 27 resolved cases for that comparison. The other three criteria had 28 resolved cases each. We calculated average pair agreement, the fraction of matching labels across reviewer pairs, and Fleiss' κ, an agreement measure adjusted for agreement expected from the label frequencies (Fleiss, 1971). We compared checker decisions with the reviewer-majority labels. Exact agreement required the same Pass, Fail, or N/A decision. For the binary comparison, Fail was positive and Pass / N/A was non-fail.

A true positive (TP) was a checker flag supported by the majority reference; a false positive (FP) was an unsupported flag. A false negative (FN) was a majority-reference failure missed by the checker; a true negative (TN) was a non-fail case under both. Precision was the fraction of checker flags supported by the reference, and recall was the fraction of reference failures flagged by the checker. The *Exact / n* column BAJ: What does this mean? reports the number of cases in which the checker's label exactly matched the majority label (Pass, Fail, or N/A), divided by the number of cases with a resolved majority decision. For example, 12/28 means exact agreement in 12 of 28 cases. The separate condition-criterion comparison used the same 10 checker-eligible cases under each disclosure definition.

Both the organizer guidance and the distributed reviewer packet planned three independent reviewers. The organizer guidance specified Fleiss' κ and a subsequent adjudication step, whereas the packet README mentioned Cohen's κ, a measure for two raters, and also anticipated adjudication. BAJ: I don't understand – why are there two conflicting approaches here? These were inconsistent analysis instructions accompanying the same three-rater design. The reported analysis used all three returned forms and Fleiss' κ; unresolved labels were compared by majority rule because a separate adjudicated reference set was not completed. The original documents and ratings are retained in the supplement.

Phase B followed Phase A on the same cases. We compared the majority issue sets and inspected the five cases flagged only in Phase B to identify which criterion accounted for the difference.

3.7. Verification and root-cause procedure

The primary audit script checked the page count, SHA-256 file identities, review-status counts, source-role descriptions, guard conditions, conflict routing, and certification wording. Primary, historical, alternative-input, and external scripts used the same released rule library for applicable checks. The primary scanner identified ten eligible guard cases. The release verifier checked the expected control labels and both conformance-test sets before accepting the corpus summaries.

After identifying documentation errors, we inspected the archived generator code to trace their causes. We linked repeated errors to a common cause when the same code explained their occurrence across pages. The supplement retains the generator excerpts and file hashes used in this inspection.

4. Results

4.1. Documentation Errors and Preserved Conflict Routing

The audit identified three causes of repeated documentation errors in the 28-page output: incorrect source-role descriptions, safeguards that lost the condition or variable checked by the code, and certification wording that appeared regardless of review status. In Table 4, *lineage typing* means assigning a role to a source excerpt, while *guard binding* means connecting an implemented condition to the correct variable and displayed formula. All six pages with explicit conflict markers retained their review routing. Table 4 separates these generator causes from their occurrences on pages.

The alternative-input runs reproduced source-role and certification-wording findings. The 29-page scenario output had one source-role finding and certification wording on all 11 unresolved pages; its one eligible zero guard passed. The 8-page challenge output had one source-role finding and certification wording on all six unresolved pages; its one eligible zero guard also passed. Marked conflicts retained their review routing in both outputs. Guard results varied with the input configuration, while the other two findings recurred.

The generator-code inspection identified three causes corresponding to the recurring findings: a source-role classifier that misidentified comments and SQL conversion types, safeguard text generated without consistently retaining the guarded variable or non-positive condition, and certification language emitted without checking the recorded review state. This explains why multiple affected pages share the same underlying mechanisms.

4.2. Checker conformance-test results

These controlled tests checked whether the program followed its specified rules on examples designed to pass or fail. The independent reviewer comparison in the next subsection addresses a different question: how many observed documentation errors the rules detected.

The checker returned the expected result for all 18 rule fixtures and 14 artifact-derived controls: 19 Fail, 11 Pass, and two N/A decisions. A correct control result included flagging a deliberately introduced defect. For example, changing the external query parameter from `query/item-query` to a header

Table 4
Repeated documentation errors and conflict routing in the current and earlier archived LEAP outputs.

Class / mechanism	Current 28-dossier build	Earlier 18-dossier build	Interpretation
Lineage typing defect	4 dossiers; 12 manifestations	Same 4 dossiers; same 12 manifestations	Same classifier defect persists across the two archived project states.
Guard-to-safeguard binding defect	5 failures / 10 eligible guards	4 failures / 9 eligible guards	Same generator weakness with one changed eligible outcome.
Certification-wording template defect	19 / 19 unresolved dossiers	12 / 12 unresolved dossiers	Same unconditional template behavior appears on every unresolved page in each build.
Conflict routing mechanism	Same 6 explicit-conflict dossiers, 6 / 6 preserved	Same 6 dossiers, 6 / 6 preserved	Intended routing behavior persists across all shared conflict cases.

or a wrong name triggered the source-role rule. Removing the displayed denominator, disabling the stated zero guard, or replacing the safeguard sentence with a repeated heading triggered the condition rule. Removing a conflict's Review Queue entry or adding certification wording beside unresolved status also triggered the relevant rule. Across the 30 binary controls, the checker missed zero introduced defects and raised zero false alarms; the other two controls returned N/A.

4.3. Reviewer Agreement and Checker Results

Across the 112 page–criterion combinations, average pair agreement was 0.634 and Fleiss' κ was 0.446, with one unresolved majority decision. Agreement was 0.553 for condition disclosure, 0.407 for source-role accuracy, 0.452 for disagreement preservation, and −0.285 for review-authority consistency. All 84 authority ratings were explicit Pass or Fail decisions. The negative authority agreement led us to reject that operational definition as a reliable assessment criterion (Fleiss, 1971).

For disagreement preservation, agreement across all 28 cases, including N/A judgments, was $\kappa = 0.452$. Restricting the analysis to the seven cases judged applicable by all three reviewers gave $\kappa = -0.167$. This comparison changed which cases were included, rather than treating N/A as a missing response. We therefore removed the broad disagreement criterion from the retained assessment dimensions and reported preservation of explicitly marked conflicts as a separate LEAP-specific routing check.

Table 5 reports the full checker comparisons. Figure 4a shows the source-role coverage directly: the checker detected two of the 16 pages identified by reviewer majority and missed the remaining 14, giving 12.5% recall. It also flagged two pages rated Pass by the majority. Two of its four flags were therefore supported, giving 50% precision. For condition disclosure, the comparison across resolved cases gave 40% precision and 100% recall; the three disputed flags are examined below.

Cases C13 and C28 illustrate the source-role coverage limitation. Their source comments describe, respectively, a cross-border transfer of personal data and unsigned trace records, but the published explanations classify those documentary comments as KPI markers, labels intended to identify KPI calculation candidates. The checker returns Pass for both dossiers, whereas all three reviewers assign source-role Fail. Its implemented patterns detect comments assigned executable roles and SQL cast types mistaken for output aliases; they do not detect these incorrect assignments between documentary roles.

The three condition checker-majority disagreements are C01 (Gross Margin Percentage), C12 (Ethylene Production Yield), and C26 (Yield Percentage). Each has the same vote split: Rater A = Pass, Rater B = Pass, and Rater C = Fail, while the checker records Fail. Evidence inspection shows a consistent definition boundary. Raters A and B use a *source-visibility* criterion: a visible source condition counts as disclosed. Rater C and the checker use a *reader-facing* criterion: the published safeguard layer must communicate the condition and bind it to the displayed representation. In C01, the source guard protects `cost == 0` while the displayed business formula denominator is Revenue and the safeguard says only "zero denominator." In C12 and C26, the source visibly contains the zero/non-positive branch while the Logic Safeguards block omits that branch.

Figure 4b shows why condition precision differed: the checker flagged the same five pages, while reviewers differed on whether a condition visible in the source also had to appear in the written explanation. Raters A and B supplied identical labels on the ten eligible pages and supported two flags; Rater C supported all five. The disputed flags were C01, C12, and C26. Precision was therefore 2/5 (40%) against Raters A and B and 5/5 (100%) against Rater C. The other five eligible pages were Pass for the checker and all reviewers, and recall was 100% under both criteria. The Rater C comparison measured agreement with the reader-facing criterion implemented by the checker. Table 7 gives the complete counts.

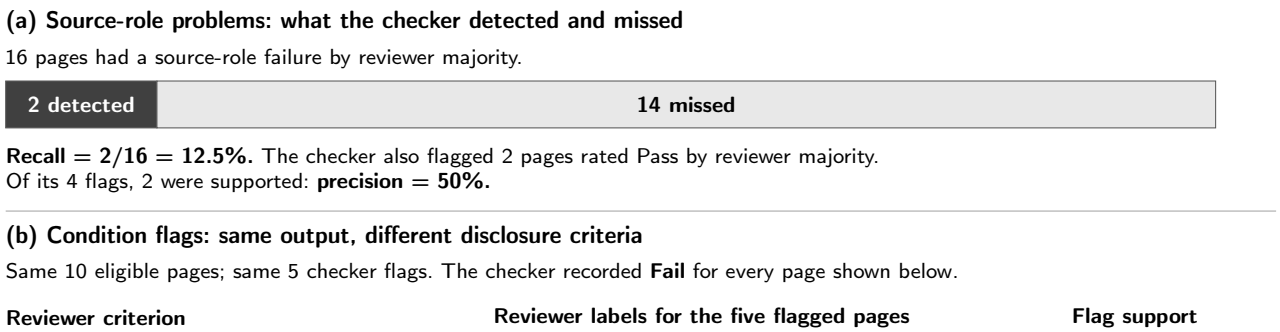


Figure 4: What the automated checks detected and where reviewer judgments differed. (a) The bar partitions the 16 pages with a reviewer-majority source-role failure into two detected and 14 missed pages. The two additional checker flags occurred on majority-Pass pages. (b) The same five condition flags are shown under the two disclosure criteria observed among reviewers on ten eligible pages. Raters A and B supported two flags; Rater C supported all five. The Rater C row measured agreement with the reader-facing criterion implemented by the checker. Tables 5–7 retain the complete counts and case details.

Table 5
Independent reviewer agreement and unconditional checker comparison. FAIL is the positive class. Precision and recall are computed against reviewer-majority labels on the fixed dossier set.

Candidate	Fleiss' κ	TP	FP	FN	TN	Precision	Recall	Exact / n	Empirical disposition
Source-role accuracy	0.407	2	2	14	10	0.50	0.125	12/28	Dimension retained; released rule fails coverage at 12.5% recall.
Condition disclosure	0.553	2	3	0	22	0.40	1.00	22/27	Dimension retained; majority comparison reflects the source-visibility criterion in three disputed cells.
Disagreement candidate	0.452	0	0	4	24	–	0.00	24/28	Broad criterion removed; agreement on commonly applicable cases was $\kappa = -0.167$; routing reported separately.
Review-authority candidate	−0.285	19	0	3	6	1.00	0.864	19/28	Criterion rejected; all 84 ratings were Pass or Fail, giving $\kappa = -0.285$.

In Phase A, reviewer majority identified issues in 18/28 pages. In Phase B, it identified issues in 23/28. The five additional cases—C03, C08, C11, C20, and C21—all arose from the authority criterion and had the same split: Raters A and C marked Fail, whereas Rater B marked Pass. These counts describe the two sequential phases; the same reviewers assessed the same cases in that fixed order.

The evaluation retained source-role accuracy and condition disclosure as assessment criteria, reported marked-conflict routing separately, and rejected the review-authority criterion. The source-role rule missed most majority-reference failures. Condition-rule decisions matched the reader-facing labels on all 10 eligible cases and differed from the source-visibility labels on three.

4.4. Comparison with source-linkage verification

We used the same Yield and Gross Margin pages for two checks. First, we checked whether each page identified and exposed the source needed to inspect the calculation. Second, we compared its source-role descriptions and safeguards with that code. We recorded a source-role failure for a wrong

Table 6
Condition cells where checker FAIL conflicts with reviewer-majority PASS. Every cell is a 2–1 split over what counts as disclosure.

Case	Dossier	Majority basis	Checker and Rater C basis	Interpretation
C01	Gross Margin Percentage	Visible `cost==0` branch counted as disclosed	Guarded Cost conflicts with displayed Revenue denominator; generic safeguard loses binding	Representation-binding criterion disagreement
C12	Ethylene Production Yield	Visible `feedstock_input_tons <= 0` branch counted as disclosed	Reader-facing safeguards omit the non-positive branch	Source-visibility versus reader-facing-disclosure disagreement
C26	Yield Percentage	Visible `total_input = 0` branch counted as disclosed	Reader-facing safeguards omit the zero-input branch	Source-visibility versus reader-facing-disclosure disagreement

Table 7
Condition-rule sensitivity on the same 10 checker-eligible cases under the two disclosure criteria observed in independent review. FAIL is the positive class. The Rater C row measures self-consistency with the reader-facing criterion implemented by the checker; the multi-reviewer analysis provides the independent comparison.

Criterion	TP	FP	FN	TN	Precision	Recall
Source visibility (Raters A and B)	2	3	0	5	0.40	1.00
Reader-facing publication (Rater C)	5	0	0	5	1.00	1.00

description of a source element and a condition failure for an omitted branch or a safeguard that referred to the wrong variable or formula. Table 8 reports the two sets of findings separately.

Both dossiers pass the source-linkage baseline. Yield's fidelity inspection identifies a source-role failure because a function docstring is described as an output alias, and a condition-disclosure failure because its reader-facing safeguards omit the return-zero branch for non-positive feedstock. Gross Margin's condition inspection identifies a representation-binding discrepancy: the safeguard loses the identity of the guarded denominator relative to the displayed formula. Its explicit formula conflict remains preserved in review routing. The condition findings in this comparison use the reader-facing criterion; the source-visibility judgments and their disagreement with that criterion are reported in Table 6. These paired observations show precisely what semantic inspection adds after source availability has been established.

Table 8
Source-linkage baseline and additional explanation-fidelity findings in the two detailed cases.

Case	Source linkage / provenance	Explanation-fidelity result
Ethylene Production Yield	Pass: source statement and executable guard retained	Source-role Fail; condition-disclosure Fail under the reader-facing criterion
Gross Margin Percentage	Pass: declared and executable representations retained with review state	Condition-disclosure Fail under the reader-facing binding criterion; explicit-conflict routing preserved

5. Detailed Case Analysis

The following two cases show how the audit works on specific calculations. For each, we examine the business meaning, the software calculation, what the generated page tells its reader, and where those descriptions disagree.

The Yield and Gross Margin cases show how an inaccurate explanation can coexist with accessible source evidence. The first case concerns a missing condition that changes the returned value; the second concerns conflicting formulas and

```
def calculate_ethylene_yield_pct(ethylene_produced_tons: float,
      feedstock_input_tons: float) -> float:
    """Calculates ethylene production yield as a percentage of feedstock
      input."""
    if feedstock_input_tons <= 0:
        return 0.0
    return (ethylene_produced_tons / feedstock_input_tons) * 100.0
```

Listing 1: Verbatim Yield source excerpt from the frozen dossier source; SHA-256 identity is retained in the supplement.

an incomplete safeguard description. Both are traced to their source statements below.

5.1. Ethylene Production Yield

Ethylene Production Yield measures ethylene output relative to the feedstock supplied to a production process. A reader expects a percentage calculated by dividing output by input and multiplying by 100. The software must also decide what to do when the input is zero or negative. In this archived example, it returns zero instead of performing the division. The audit examined whether the generated explanation told readers about that branch.

Listing 1 shows the Python function used to calculate Yield, including the return-zero condition. The dossier displays the positive-input ratio as its Formal Formula. The frozen source uses `feedstock_input_tons <= 0` and returns 0.0 on that branch. The reader-facing Logic Safeguards block omits this non-positive-input behavior. For the docstring shown in Listing 1, the generated lineage text states “Output alias: maps this expression to a in the result set.” BAJ: I don’t understand – what is "a"? The word “a” appears literally in the generated page; it is not the name of a variable or output in this Python function. The generator incorrectly treats a descriptive docstring as an output alias and produces a nonsensical alias description. Direct source comparison establishes the source-role error. The condition finding illustrates the criterion boundary exposed by independent review: two reviewers counted the visible source branch as disclosure, whereas one reviewer and the checker required the reader-facing safeguard layer to state the branch.

5.2. Gross Margin Percentage

Gross Margin Percentage normally expresses the share of revenue remaining after costs. Unlike Yield, this case concerns two conflicting calculations: the business formula printed in the dossier and the expression executed by the software. Showing both formulas makes the disagreement visible before examining the resulting numbers.

Let R be revenue and C be cost. The declared Gross Margin formula D and the implemented expression I differ:

$$D = 100\frac{R-C}{R}, \qquad I = 100\frac{R}{C}.$$

For $R > 0$ and $C > 0$,

$$\begin{aligned} I - D &= 100\left(\frac{R}{C} + \frac{C}{R} - 1\right) \\ &= 100\left(1 + \frac{(R-C)^2}{RC}\right) \geq 100. \end{aligned}$$

Thus the implemented expression exceeds the declaration by at least 100 percentage points throughout the strictly positive domain, with equality in this lower bound when $R = C$. At positive revenue and zero cost, the declaration evaluates to 100 while the implementation guard returns zero; at zero revenue and positive cost, the declaration is undefined while the implementation returns zero.

The dossier correctly routes the explicit disagreement through *Needs Review* and a Review Queue. Its safeguard text nevertheless describes a generic zero denominator even though the declared and executable representations use different denominators, and certification-oriented language appears alongside unresolved status. The page preserves explicit-conflict routing while also exhibiting a guard-binding discrepancy and certification wording beside an unresolved review state.

6. Discussion

6.1. Interpretation of the Core Findings

The Introduction asked whether a generated KPI page correctly explained the calculation it linked to. The Yield and Gross Margin cases showed why source access and explanation accuracy required separate checks: the code remained available while the page omitted a behavior-changing branch or lost the identity of the guarded denominator. Tracing those errors to the generator answered RQ1 by locating where the explanation went wrong. Recurring errors across archived outputs and alternative inputs were traced to shared generation rules; the changing guard outcomes showed that the same generator could describe some inputs correctly and others incompletely (RQ2).

The reviewer comparison separated agreement on an assessment criterion from the coverage of its automated rule (RQ3–RQ4). Source-role accuracy and condition disclosure received positive reviewer agreement, yet the source-role rule detected only two of 16 majority-reference failures. Agreement on what should be checked and success in detecting it were therefore distinct results. The other candidates required different treatment. Negative agreement led us to reject review-authority consistency. Negative agreement on commonly applicable disagreement cases led us to retain only the narrower marked-conflict routing check. Preserving a marked conflict and accurately explaining a calculation remained separately assessable properties.

6.2. Relation to Prior Research

The observed errors align with code–documentation consistency research. Xu et al. (2024) developed a method for detecting inconsistent code comments that addressed noise and labeling errors in training data. Zhou et al. (2017)

compared API documentation with code to detect defects in parameter constraints and exception directives. The LEAP findings connect these concerns to generated KPI pages: the SQL example misdescribed a source element, while Yield omitted an implemented condition. Our study extended the comparison to business-facing formula, explanation, and safeguard fields, then traced errors to the generator. Separately assessing reviewer agreement and checker coverage exposed both ambiguity in the criterion and gaps in the implemented patterns.

Traceability and inconsistency-management research help explain what the retained evidence contributed. Mäder and Gotel (2012) studied maintenance of traceability links as software models changed; our audit examined explanations after the relevant source links were already present. The links made the disputed calculation inspectable. Nuseibeh et al. (2001) described retaining and managing known inconsistencies as part of software development. Gross Margin provided a concrete instance: both formulas and their review routing remained visible, making their disagreement available for inspection. The inaccurate safeguard on that same page showed why preserving a conflict and explaining the associated calculation are complementary responsibilities.

6.3. Why the Checks Disagreed with Reviewers

The checker and a reviewer can disagree because the rule misses a type of error or because they use different definitions of what counts as adequate documentation. The source-role and condition findings show these two reasons separately.

Figure 3 shows how a generator pattern produced a source-role error: a generic `AS` match selected the SQL conversion type instead of the final output alias. The checker targeted specified patterns of this kind and returned all expected control decisions. Cases C13 and C28 exposed a different gap: documentary comments were called KPI markers, whereas the checker targeted comments described as executable code and cast types described as aliases. Those cases explain how correct execution of prepared controls coexisted with low recall on reviewer-identified errors (Figure 4a). They identify a missing class of role distinctions for subsequent rule development.

The condition result arose through a different mechanism (Figure 4b). The checker flagged the same five pages under both comparisons. Raters A and B accepted source visibility as disclosure, whereas Rater C required the written safeguard to state the condition and its variable. Consequently, the same three flags changed from unsupported to supported when the reference criterion changed, producing precision of 40% and 100%. The difference came from how disclosure was judged. For Yield, the disputed information was present in code but absent from safeguards; for Gross Margin, the safeguard lost the relationship between the guarded variable and the displayed denominator. These cases explain why an explicit disclosure definition is needed to interpret checker performance.

6.4. Practical and Scientific Implications

The worked cases provide concrete review questions. For Yield, a documentation maintainer should check that the explanation states the return-zero behavior for non-positive feedstock. For Gross Margin, the maintainer should identify the denominator being guarded and preserve both conflicting expressions and the review state. The source excerpt, generated description, and located discrepancy form a reviewable record connecting the business explanation to implementation evidence. Generator tracing then identifies the rule or template to inspect when the same error appears on several pages. This is a practical use of the evidence produced by the audit.

For subsequent checker development, the findings support two separate priorities: extend source-role patterns to the documentary distinctions illustrated by C13/C28, and specify the condition-disclosure criterion before collecting comparison judgments. A subsequent evaluation should test revised rules on additional codebases and retain separate measurements of reviewer agreement, rule conformance, and detection coverage. These measures address different questions about an audit. Comparative documentation studies and cross-role interpretation studies would then address correction effort, business readers' understanding, and joint review with developers as distinct outcomes.

6.5. Study Limitations

The 28 pages came from one author-developed system; shared generation rules produced repeated errors, so page counts describe this output rather than independent generators. The earlier and alternative-input outputs extended the within-system comparison, and the four FastAPI tutorial examples established adapter execution. Broader recurrence requires evidence from other generators. The three-reviewer majority was a comparison reference, with overall $\kappa = 0.446$, rather than a separately adjudicated final set. Reviewer disagreement therefore affects estimates measured against that reference. Original N/A labels indicated inapplicability; restricting analysis to common applicability changed the case set and the disagreement estimate. The source-visibility and reader-facing criteria were identified in the returned judgments, so the Rater C comparison measures agreement with the criterion implemented by the checker.

The fixed phase order and inclusion of both phase instructions in one packet prevented separation of checklist effects from familiarity or prior rubric access. The increase from 18/28 to 23/28 flagged pages was descriptive; all five additions arose from the rejected authority criterion. Root-cause inspection traced existing output to generator code; a prospective correction-and-retest evaluation remains future work. The routing check covered explicitly marked conflicts, and the condition rule covered comparisons with zero. Near-zero tolerances and function-domain cases require separate rules: real square roots require nonnegative inputs, logarithms positive inputs, and inverse sine/cosine inputs in $[-1, 1]$; inverse tangent accepts all real inputs, and positive-base exponentiation allows negative exponents. The recorded

outcomes concern documentation and checking. Human comprehension, correction time, and collaborative benefit require dedicated outcome studies.

7. Conclusion

We audited 28 source-linked KPI documentation pages and found descriptions that assigned source elements the wrong role or omitted conditions that changed the calculation. Comparing checker decisions with three reviewers' judgments showed which errors the implemented rules detected and missed. Inspection of the generator code traced repeated discrepancies to their causes. Reviewer agreement supported source-role accuracy and condition disclosure as the two retained assessment criteria.

The source-role checker achieved 12.5% recall. On the same 10 eligible condition cases, precision was 40% under source visibility and 100% under reader-facing disclosure, with 100% recall under both. The 100% comparison measured agreement with the criterion the checker implemented. The broad disagreement criterion was narrowed to a separate check of explicit-conflict routing, which was preserved in 6/6 cases; negative reviewer agreement led us to reject the review-authority criterion. The initial majority issue set contained 18 of 28 pages.

The study contributed a source-grounded procedure for checking generated KPI explanations and an evaluation that distinguished reviewer support for assessment criteria from the coverage of automated rules. The worked cases located incorrect descriptions in their source evidence and traced repeated errors to generator logic. Subsequent evaluations can test revised checking rules on additional codebases using a disclosure criterion specified before reviewer assessment.

Data and artifact availability

The frozen technical supplement contains the 28 published pages, historical and alternative-input outputs, source and page hashes, audit rules, control labels, scripts, generator excerpts, and the Yield and Gross Margin sources. The unchanged independent-evaluation archive contains the returned-form derivatives, normalized analysis forms, case mapping, reviewer agreement and checker comparisons, and analysis scripts. Files in that archive's raw directory are intermediate exports: 94 original N/A labels appear there as blanks. The accompanying reporting-correction supplement preserves the original obligation responses, a cell-by-cell reconciliation, and both protocol documents. The reconciliation restores the original N/A values and matches all 336 labels used in the analysis.

Declaration of competing interest

The authors declare that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During preparation of this work, the author used Elsevier LeapSpace to support literature discovery, manuscript organization, language refinement, and consistency checking. The authors take responsibility for the content of the article.

References

Aghajani, E., Nagy, C., Vega-Márquez, O.L., Linares-Vásquez, M., Moreno, L., Bavota, G., Lanza, M., 2019. Software documentation issues unveiled, in: Proceedings of the 41st International Conference on Software Engineering, pp. 1199–1210. doi:10.1109/ICSE.2019.00122.

Brush, M.H., Shefchek, K., Haendel, M., 2016. SEPIO: A semantic model for the integration and analysis of scientific evidence. CEUR Workshop Proceedings, Vol. 1747, International Conference on Biomedical Ontology. URL: https://ceur-ws.org/Vol-1747/IT605_ICBO2016.pdf.

Diamantini, C., Khan, T., Potena, D., Storti, E., 2025. Semantic models of performance indicators: A systematic survey. ACM Computing Surveys 57. doi:10.1145/3719291.

Domínguez, E., Pérez, B., Rubio, Á.L., Zapata, M.A., 2025. A flexible framework to ensure traceability, consistency, and propagation of KPIs evolution. Journal of Software: Evolution and Process 37, e70004. doi:10.1002/smr.70004.

Doxygen, n.d. Documenting the code. Doxygen Manual. URL: https://www.doxygen.nl/manual/docblocks.html. accessed 7 October 2026.

Eghbali, A., Liu, Z., Pradel, M., 2026. Natural language-focused software engineering via code-documentation equivalence. Proceedings of the ACM on Software Engineering 3, 4644–4665. doi:10.1145/3808211.

FastAPI Project, 2026. FastAPI tutorial source examples. GitHub repository, commit 33d411dbc3236275dd64d200bfe18d5d60a49b2e. URL: https://github.com/fastapi/fastapi/tree/33d411dbc3236275dd64d200bfe18d5d60a49b2e/docs_src.

Fleiss, J.L., 1971. Measuring nominal scale agreement among many raters. Psychological Bulletin 76, 378–382. doi:10.1037/h0031619.

Gao, X., 2026. Executable code knowledge: Code as a native, validation-carrying knowledge representation for AI coding agents. arXiv preprint arXiv:2608.16295 URL: https://arxiv.org/abs/2608.16295.

Giglio, M., Tauber, R., Nadendla, S., Munro, J., Olley, D., Ball, S., Mitraka, E., Schriml, L.M., Gaudet, P., Hobbs, E.T., Erill, I., Siegele, D.A., Hu, J.C., Mungall, C., Chibucos, M.C., 2019. ECO, the evidence & conclusion ontology: community standard for evidence information. Nucleic Acids Research 47, D1186–D1194. doi:10.1093/nar/gky1036.

Knuth, D.E., 1984. Literate programming. The Computer Journal 27, 97–111. doi:10.1093/comjnl/27.2.97.

de Macedo, S.O., da Costa, R.M., 2026. Reversa: A reverse documentation engineering framework for converting legacy software into operational specifications for AI agents. arXiv preprint arXiv:2605.18684 URL: https://arxiv.org/abs/2605.18684.

Mäder, P., Gotel, O., 2012. Towards automated traceability maintenance. Journal of Systems and Software 85, 2205–2227. doi:10.1016/j.jss.2011.10.023.

Maté, A., Trujillo, J.C., Mylopoulos, J., 2017. Specification and derivation of key performance indicators for business analytics: A semantic approach. Data & Knowledge Engineering 108, 30–49. doi:10.1016/j.datak.2016.12.004.

Missier, P., Belhajjame, K., Cheney, J., 2013. The W3C PROV family of specifications for modelling provenance metadata, in: Proceedings of the 16th International Conference on Extending Database Technology, pp. 773–776. doi:10.1145/2452376.2452478.

Nuseibeh, B., Easterbrook, S., Russo, A., 2001. Making inconsistency respectable in software development. Journal of Systems and Software 58, 171–180. doi:10.1016/S0164-1212(01)00036-X.

OpenAPI Initiative, 2026. OpenAPI specification, version 3.1. OpenAPI Initiative specification. URL: https://spec.openapis.org/oas/v3.1.

Oracle, n.d. JavaDoc documentation comment specification for the standard doclet (JDK 26). Java Platform, Standard Edition Documentation. URL: https://docs.oracle.com/en/java/javase/26/docs/specs/javadoc/doc-comment-spec.html. accessed 7 October 2026.

Psallidas, F., Agrawal, A., Sugunan, C., Ibrahim, K., Karanasos, K., Camacho-Rodríguez, J., Floratou, A., Curino, C., Ramakrishnan, R., 2023. OneProvenance: Efficient extraction of dynamic coarse-grained provenance from database query event logs. Proceedings of the VLDB Endowment 16, 3662–3675. doi:10.14778/3611540.3611555.

Roldán-García, M.d.M., García-Nieto, J.M., Maté, A., Trujillo, J.C., Aldana-Montes, J.F., 2021. Ontology-driven approach for KPI meta-modelling, selection and reasoning. International Journal of Information Management 58, 102018. doi:10.1016/j.ijinfomgt.2019.10.003.

Sadiq, M.A., Langat, P.K., Neupane, A., 2026. Enterprise spatial data provenance knowledge infrastructure. ISPRS International Journal of Geo-Information 15, 182. doi:10.3390/ijgi15050182.

The Rust Project Developers, n.d. Documentation tests. The rustdoc Book. URL: https://doc.rust-lang.org/rustdoc/write-documentation/documentation-tests.html. accessed 7 October 2026.

Wen, F., Nagy, C., Bavota, G., Lanza, M., 2019. A large-scale empirical study on code-comment inconsistencies, in: Proceedings of the 27th International Conference on Program Comprehension, pp. 53–64. doi:10.1109/ICPC.2019.00019.

World Wide Web Consortium, 2013. Constraints of the PROV data model. W3C Recommendation. URL: https://www.w3.org/TR/prov-constraints/.

Xu, Z., Guo, S., Wang, Y., Chen, R., Li, H., Li, X., Jiang, H., 2024. Code comment inconsistency detection based on confidence learning. IEEE Transactions on Software Engineering 50, 598–617. doi:10.1109/TSE.2024.3358489.

Zhou, Y., Gu, R., Chen, T., Huang, Z., Panichella, S., Gall, H.C., 2017. Analyzing APIs documentation and code to detect directive defects, in: Proceedings of the 39th IEEE/ACM International Conference on Software Engineering, pp. 27–37. doi:10.1109/ICSE.2017.11.